Notes on Machine Learning

Course of Machine Learning

Master Degree in Computer Science

University of Rome "Tor Vergata"

Giorgio Gambosi

Academic Year: 2025-2026

Chapter 1

Foundations of Machine Learning

Machine learning rests on a deceptively simple observation: data is rarely random. Whether we're analyzing images, text, sensor measurements, or human behavior, the data we collect contains hidden patterns—structures that reflect the underlying process that generated the data. The fundamental challenge, and the promise, of machine learning is to automatically discover these patterns from observations, without being explicitly programmed with the rules that govern the data.

This discovery happens through learning from examples. We begin with a collection of observations, each a snapshot of some phenomenon we wish to understand. From these examples, we construct **models**—mathematical abstractions that capture the relationships and patterns present in the data. These models then serve as tools for prediction, understanding, and decision-making on new, previously unseen data.

Three Paradigms of Learning

Machine learning encompasses three fundamentally different learning scenarios, each posing its own challenges and opportunities:

Supervised Learning

In supervised learning, we have pairs of observations: input features and corresponding target values. Our goal is straightforward: learn the relationship between inputs and targets so we can predict targets for new inputs.

Imagine we have a dataset of houses with features like square footage, number of bedrooms, and location. If each house also has a known selling price, we're in a supervised setting. We want to learn a function that maps features to prices, so we can estimate the price of a new house from its characteristics.

Supervised learning splits into two categories depending on the nature of the target:

Regression The target is a real number (continuous): predicting prices, temperatures, stock values, or any quantity on a numerical spectrum.

Classification The target is a discrete category: predicting whether an email is spam or not, recognizing handwritten digits (0-9), diagnosing diseases, or determining sentiment in text.

In both cases, we formalize the learning process by defining a model—either a function $y(\mathbf{x})$ that directly outputs a prediction, or a probability distribution $p(t|\mathbf{x})$ that quantifies our uncertainty about the target value.

We then use the training data to adjust this model's parameters so it captures the underlying input-output relationship as accurately as possible.

Unsupervised Learning

Unsupervised learning operates in a more exploratory setting. We have data, but no explicit targets guiding us. Instead, our goal is to uncover the inherent structure within the data—to understand its nature and organize it in meaningful ways.

Unsupervised learning manifests in several important forms. **Clustering** groups similar items together, automatically revealing natural categories or segments in the data without being told what those categories should be. **Density estimation** models the underlying probability distribution that generated the data, revealing where data points are concentrated and how they're distributed across the feature space. **Dimensionality reduction** projects high-dimensional data onto lower-dimensional representations while preserving essential information—this is crucial when data has hundreds or thousands of features but lies primarily in a much lower-dimensional subspace.

A particularly powerful application of unsupervised learning is **generative modeling**, where we learn to generate new synthetic data points that appear to come from the same underlying process as our training data. Large language models, diffusion models, and GANs exemplify this capability: after observing enough examples, they can generate new text, images, or other content that feels authentic and novel. This requires the model to have truly learned the deep structure of the data distribution.

Even without explicit targets, unsupervised learning still relies on models. These models represent our understanding of the data structure and enable us to perform several valuable tasks: they allow us to identify outliers and anomalies (points that deviate significantly from the learned pattern), they facilitate visualization and interpretation of complex, high-dimensional data, and they provide the foundation for feature engineering—the process of creating new, more informative representations of the data.

Reinforcement Learning

Reinforcement learning represents a fundamentally different learning paradigm. Rather than working with a fixed dataset, we learn through dynamic interaction with an environment. An agent repeatedly chooses actions, observes the consequences, and adjusts its behavior to maximize cumulative reward over time.

Concretely, at each step the agent observes the current **state** of the environment and must select an **action** from its available options. The environment responds by providing two pieces of feedback: a **reward** indicating the immediate benefit of that action, and a new state representing the updated environment. The agent's goal is to discover an optimal **policy**—a strategy that maps states to actions in a way that maximizes expected cumulative reward.

Mathematically, if we denote the policy as $\pi : S \to A$ (mapping states to actions), the optimization objective is:

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\, \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where $\gamma \in [0, 1]$ is a discount factor balancing immediate versus future rewards, and r_t is the reward at step t.

Reinforcement learning has proven remarkably successful in domains where trial-and-error learning makes sense: game playing (where AlphaGo defeated world champions), robotics (learning motor skills and navigation), and autonomous systems (optimizing resource allocation). The central tension in reinforcement learning is the exploration-exploitation tradeoff: we must try new actions to discover potentially better strategies, but we also want to exploit actions we know are good to maximize reward.

Because reinforcement learning operates in an interactive, sequential decision-making framework quite different from supervised and unsupervised learning, it employs distinct mathematical techniques and algorithms. For this reason, we shall set it aside and focus primarily on supervised and unsupervised learning.

Data Representation

At the heart of any machine learning approach lies a fundamental choice: how to represent data. Real-world entities—images, documents, customers, transactions, biological samples—must be converted into feature vectors that machine learning algorithms can process.

A **feature vector** $\mathbf{x} = (x_1, x_2, \dots, x_d)$ is a collection of d numerical measurements describing some object. These features are the raw material from which models are built. The choice of which features to include is consequential: it affects both the quality of predictions and the computational efficiency of learning.

Consider an image classification task. We could represent each pixel intensity as a feature, yielding millions of features—but this is often wasteful and noisy. Alternatively, we might extract higher-level features like edge orientations or textures. For medical diagnosis from clinical data, we might include measurements like blood pressure, cholesterol levels, and test results. For text analysis, we might count word frequencies or learn semantic embeddings. The same underlying object can be represented in innumerable ways, and the choice profoundly shapes the learning problem.

This is why **feature engineering**—the process of defining new features through functional combinations of existing ones, selecting the most informative subset, or learning better representations entirely—is often as important as the choice of learning algorithm itself. Sometimes domain expertise guides this process: a cardiologist knows which measurements matter for heart disease. Other times, sophisticated unsupervised learning techniques discover the most informative lower-dimensional representations automatically.

The Learning Process

Regardless of which learning paradigm we adopt, the fundamental process is similar. We begin with a **training set** $\mathcal{T} = (\mathbf{X}, \mathbf{t})$ containing n examples. In supervised learning, each example pairs an input feature vector $\mathbf{x}_i$ with its target value t_i. In unsupervised learning, we have only the feature vectors $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$.

The learning algorithm processes this training data to construct a model—a function or probability distribution that captures the patterns present in the training data. But here's the crucial point: we don't care how well the model fits the training data. What matters is how well it generalizes to new, unseen data.

This distinction between training and test performance reveals the core challenge in machine learning: **generalization**. A model that memorizes the training data perfectly will fail on new examples it hasn't seen. Conversely, a model that's too simple may miss important patterns. We seek the middle ground—models that capture genuine structure in the data-generating process without overfitting to noise or idiosyncrasies in our particular training sample.

Achieving good generalization requires understanding several interrelated concepts. The **capacity** or **complexity** of a model determines how flexible it can be: more complex models can fit more intricate patterns but risk overfitting. The **bias-variance tradeoff** reflects the tension between underfitting (models too simple to capture the true pattern) and overfitting (models that adapt too closely to training noise). **Regularization** techniques penalize model complexity, encouraging simpler solutions that generalize better.

Throughout all these learning scenarios, models serve as the bridge between data and prediction. A model embodies what we've learned from the training data—our best current understanding of the underlying patterns and relationships.

In supervised learning, the model learns the input-output relationship, enabling predictions for new inputs. In unsupervised learning, the model captures the structure of the data distribution, enabling clustering, density estimation, or synthesis of new examples. The specific form a model takes depends on the task: it might be a simple linear function, a complex neural network, a probabilistic distribution, a tree-based ensemble, or something else entirely.

What unites all these diverse approaches is their shared goal: to construct a model from data that captures genuine patterns while remaining general enough to perform well beyond the training set. This is machine learning at its essence—learning models from data that transfer knowledge to new situations.

Formal Definition of a Supervised Machine Learning Task

At its core, a supervised learning task is defined over two essential domains: the **domain set** $\mathcal{X}$ and the **label set** $\mathcal{Y}$.

The domain set $\mathcal{X}$ represents the collection of objects we wish to label or make predictions about. Each object $\mathbf{x} \in \mathcal{X}$ is typically modeled as an array of **features**—think of features as measurable properties or attributes of the object. Formally, we represent an object as $\mathbf{x} = (x_1, x_2, \dots, x_d)$, where d denotes the **dimensionality**, or the total number of features describing the object. In more advanced scenarios, objects might have more intricate structures such as sequences or graphs, but for most standard problems, this feature representation suffices.

The label set $\mathcal{Y}$ contains all possible output values that we might assign to objects in $\mathcal{X}$. The nature of this set fundamentally determines what kind of learning problem we're facing. If $\mathcal{Y}$ is continuous (think of all real numbers in a certain range), we're dealing with a **regression** task, where we predict numerical values. Conversely, when $\mathcal{Y}$ is discrete (a finite set of distinct categories), we have a **classification** task. Within classification, we

further distinguish between **binary classification** when $|\mathcal{Y}| = 2$ (just two possible categories) and **multi-class classification** when $|\mathcal{Y}| > 2$.

To learn how to map objects to their labels, the learning algorithm $\mathcal{A}$ receives a **training set** $\mathcal{T}$, which is a collection of item-label pairs: $\mathcal{T} = \{(\mathbf{x}_1, t_1), \dots, (\mathbf{x}_n, t_n)\}$. These pairs are usually organized into a **feature matrix** $\mathbf{X}$ containing all the objects as rows, and a **target vector** $\mathbf{t}$ containing all the corresponding labels:

$$\mathbf{X} = \begin{pmatrix} - & \mathbf{x}_1 & - \\ & \vdots & \\ - & \mathbf{x}_n & - \end{pmatrix}, \quad \mathbf{t} = \begin{pmatrix} t_1 \\ \vdots \\ t_n \end{pmatrix}$$

The ultimate goal of the learning algorithm is to return a **prediction rule**—also called a **classifier** or **regressor** depending on the task—which is a function $h : \mathcal{X} \mapsto \mathcal{Y}$. This function should be able to predict a label y for any new object $\mathbf{x}$ it encounters.

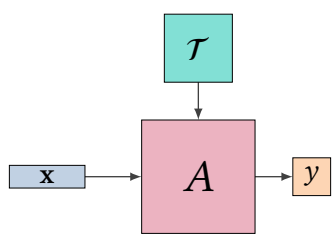
When making predictions, we have two main strategies. The first is **direct target value prediction**, where the algorithm outputs a single predicted label y by directly computing $h(\mathbf{x})$. The second approach is **probability distribution prediction**, where the algorithm outputs a probability distribution over $\mathcal{Y}$, providing an estimated probability $p(y|\mathbf{x})$ for each possible label. If we need to commit to a single prediction in this case, we typically apply an independent decision rule, such as selecting the label with the highest probability.

Three Approaches to Deriving a Predictor

There are fundamentally three different strategies for constructing a prediction function, each with its own characteristics and use cases.

Direct Prediction Computation

In the simplest approach, we apply a predefined algorithm A directly at prediction time. Rather than learning a fixed function from the training set, the algorithm computes a fresh prediction for each new item by taking into account the entire training set. Formally, the algorithm computes $y = h(\mathbf{x}, \mathcal{T})$—notice how both the new item $\mathbf{x}$ and the entire training set appear as inputs. This approach performs no explicit learning phase; instead, the training examples are stored and referenced during prediction.

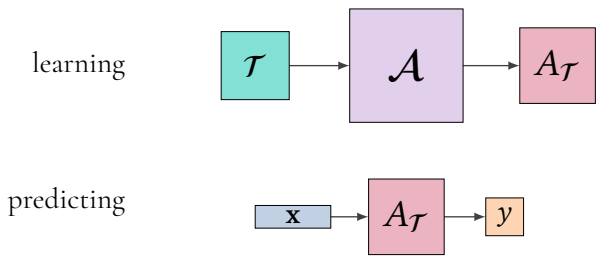


A classic example is the ***k*-nearest neighbors** algorithm. When we want to predict the class for a new item $\mathbf{x}$, we find the k objects in the training set $\mathbf{X}$ that are closest to $\mathbf{x}$ according to some distance measure. The predicted class is simply the majority class among these k nearest neighbors. This approach is appealingly simple: it requires

no explicit learning phase and can adapt naturally to different regions of the input space. Also, note that no model of the data is introduced to describe the properties of the data itself useful for the prediction.

Model-Based Learning Approach

The second approach is far more typical in modern machine learning. Here, we don't defer all computation to prediction time. Instead, we define a **hypothesis class** $\mathcal{H}$—a predefined family of functions mapping $\mathcal{X}$ to $\mathcal{Y}$—and use a learning algorithm $\mathcal{A}$ to select the single best function from this class. The algorithm derives a specific function $h_{\mathcal{T}} \in \mathcal{H}$ that best (according to some criterion) explains the training examples in $\mathcal{T}$. Once we have this function, predictions for new items are straightforward: we simply apply $h_{\mathcal{T}}(\mathbf{x})$.



In many cases, the hypothesis class $\mathcal{H}$ consists of **parametric functions**—functions with the same underlying structure but differing in their parameter values. We can denote such a class as $\mathcal{H}_{\boldsymbol{\theta}}$, where $\boldsymbol{\theta} = (\theta_1, \dots, \theta_m)$ is a vector of parameters. The learning task then reduces to finding the optimal parameter values that best fit the training data.

Linear regression provides an excellent illustration. In linear regression, we assume that the target value can be predicted as a weighted sum of the features, plus a constant term:

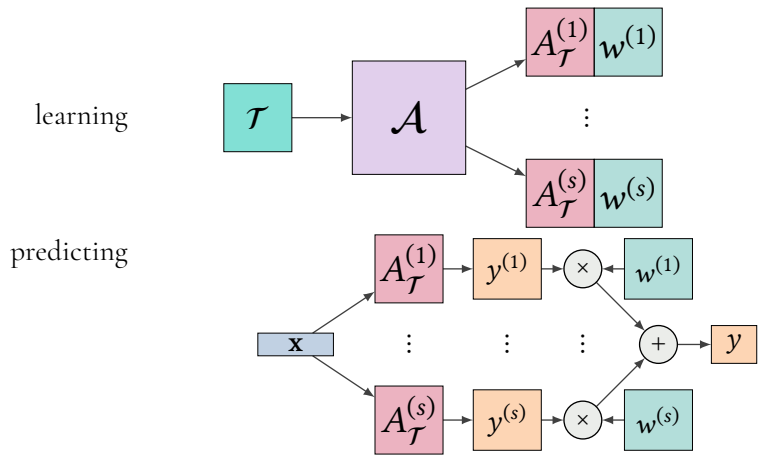
$$y = \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{w}^T \overline{\mathbf{x}}$$

where $\mathbf{w} = (w_0, w_1, \dots, w_d)^T$ is the parameter vector and $\overline{\mathbf{x}} = [1, x_1, \dots, x_d]^T$ includes a constant component for the bias term. The hypothesis class here is the set of all $(d+1)$-dimensional linear functions, and the learning algorithm must find the weights $w_0, w_1, \dots, w_d$ that minimize prediction error on the training set.

Ensemble Learning Approach

The third approach takes a different philosophy: instead of searching for a single best predictor, we construct multiple predictors and combine their predictions. This **ensemble** strategy often yields better performance than any individual predictor, as different models can capture different patterns in the data.

The process unfolds in three stages. First, we derive a collection of s different predictors $A_{\mathcal{T}}^{(1)}, \dots, A_{\mathcal{T}}^{(s)}$ from the training set, each computing a function $h_{\mathcal{T}}^{(i)} : \mathcal{X} \to \mathcal{Y}$. These predictors might differ in their structure, their parameters, or the subset of training data they use. Second, we derive a weight $w^{(i)}$ for each predictor, reflecting its estimated quality or reliability. Third, when we need to make a prediction for a new item $\mathbf{x}$, we gather the predictions $y^{(i)} = h_{\mathcal{T}}^{(i)}(\mathbf{x})$ from each individual predictor and combine them using the weights.



The combination method depends on the task at hand. For regression, we typically compute a weighted average of the individual predictions:

$$h(\mathbf{x}) = \sum_{i=1}^{s} w^{(i)} h^{(i)}(\mathbf{x})$$

where the weights satisfy $\sum_{i=1}^{s} w^{(i)} = 1$ and $w^{(i)} \geq 0$ for all i, ensuring that the result is interpretable as an average. For classification tasks, a natural approach is **weighted voting**, where we select the class that receives the most weight:

$$h(\mathbf{x}) = \operatorname*{argmax}_{y \in \mathcal{Y}} \sum_{i=1}^{s} w^{(i)} \mathbb{1}[h^{(i)}(\mathbf{x}) = y]$$

where $\mathbb{1}[\cdot]$ is the indicator function that equals one if the predicate is true and zero otherwise. This aggregates the votes from all predictors, weighting each vote by the predictor's reliability.

An important variant of ensemble learning is **fully Bayesian** prediction. Rather than working with a discrete set of predictors, we derive a probability distribution on the parameters values from the dataset, which is used to effectively average predictions over all possible parameter values. Mathematically, the weighted sum becomes an integral over the parameter space, allowing us to naturally account for uncertainty in the model parameters.

These three approaches represent fundamentally different philosophies about when and how to process the training data. In direct prediction, we process both the new item and the entire training set at prediction time, performing all computation on demand. In model-based learning, we invest computational effort upfront during a learning phase to produce a compact predictor that can be applied efficiently later. In ensemble methods, we sidestep the question of finding a single best model by embracing multiple perspectives, each encoded in a different predictor, and letting them collectively decide the final prediction. Each approach has advantages: direct methods are flexible and can adapt locally to the data; model-based methods are efficient and interpretable; ensemble methods are robust and often achieve superior predictive performance.

Probabilistic Framework for Machine Learning

To build a rigorous foundation for understanding machine learning, we need to think probabilistically about how data arises. The core assumption is that both the objects we observe and their labels come from some underlying

stochastic process which results into a probability distribution on the labels domain.

We then assume that the objects in any training set are sampled (we assume independently) from an unknown probability distribution p_M over the domain $\mathcal{X}$. For each object $\mathbf{x} \in \mathcal{X}$, the quantity $p_M(\mathbf{x})$ represents the probability that $\mathbf{x}$ appears in our dataset. Similarly, we assume that labels are not deterministically assigned to objects but rather arise from a conditional probability distribution $p_C(t|\mathbf{x})$. This captures the realistic scenario where multiple legitimate labels might exist for the same object, whether due to labeling noise, inherent ambiguity, or subjective judgment. Together, these assumptions induce a joint distribution $p(\mathbf{x}, t) = p_M(\mathbf{x})p_C(t|\mathbf{x})$ over object-label pairs.

For simplicity and intuition, we will often make the simpler assumption that there exists an unknown **ground truth function** $f : \mathcal{X} \to \mathcal{Y}$ such that labels are deterministically generated as $t = f(\mathbf{x})$. This can be viewed as a special case where $p_C(t|\mathbf{x})$ places all its probability mass on the value $f(\mathbf{x})$. While this deterministic view is cleaner conceptually, the more general probabilistic setting better reflects real-world scenarios where perfect labels don't always exist.

Risk and Loss

Once we have a predictor $h : \mathcal{X} \to \mathcal{Y}$, we need a way to measure how well it performs. The foundation of this measurement is the **loss function** $L : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$, which quantifies the penalty incurred when we predict y but the true label is t. For example, in binary classification with the **0-1 loss**, we pay a penalty of 1 for any wrong prediction and 0 for correct ones:

$$L(y, t) = \mathbb{1}[y \neq t]$$

For a single object $\mathbf{x}$, the **prediction risk** $\mathcal{R}(y, \mathbf{x})$ measures the expected loss under the true label distribution. In the deterministic setting where labels come from a function f, this is simply:

$$\mathcal{R}_f(y, \mathbf{x}) \triangleq L(y, f(\mathbf{x}))$$

In the general probabilistic case, where we only know the conditional distribution $p_C(t|\mathbf{x})$, the prediction risk becomes an expectation over all possible labels:

$$\mathcal{R}_{p_C}(y, \mathbf{x}) \triangleq \mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})}[L(y, t)] = \int_{\mathcal{Y}} L(y, t) \cdot p_C(t|\mathbf{x})\, dt$$

For discrete label sets like those in classification, this integral becomes a sum:

$$\mathcal{R}_{p_C}(y, \mathbf{x}) \triangleq \sum_{t \in \mathcal{Y}} L(y, t) \cdot p_C(t|\mathbf{x})$$

The key insight is that lower prediction risk means better alignment between our prediction and the true label distribution.

To evaluate a predictor h overall, we must aggregate the prediction risk across all possible objects according to their probability of occurrence. This leads to the notion of **expected risk** (or **true risk**), which measures the average loss we would incur if we applied h to infinitely many examples drawn from the true distribution:

$$\mathcal{R}_{p_M,f}(h) \triangleq \mathop{\mathbb{E}}_{\mathbf{x}\sim p_M}[L(h(\mathbf{x}), f(\mathbf{x}))] = \int_{\mathcal{X}} L(h(\mathbf{x}), f(\mathbf{x})) \cdot p_M(\mathbf{x})\, d\mathbf{x}$$

In the probabilistic setting:

$$\mathcal{R}_p(h) \triangleq \mathop{\mathbb{E}}_{(\mathbf{x},t)\sim p}[L(h(\mathbf{x}), t)] = \int_{\mathcal{X}} \int_{\mathcal{Y}} L(h(\mathbf{x}), t) \cdot p_M(\mathbf{x}) \cdot p_C(t|\mathbf{x})\, d\mathbf{x}\, dt$$

When we use 0-1 loss, the expected risk simplifies to the probability of making an error:

$$\mathcal{R}_{p_M,f}(h) = \mathop{\mathbb{P}}_{\mathbf{x}\sim p_M}[h(\mathbf{x}) \neq f(\mathbf{x})]$$

This is the most intuitive metric: it directly tells us how often our predictor is wrong on randomly sampled data.

Unfortunately, in machine learning the true distributions p_M and p_C (or equivalently, the function f) are by hypothesis unknown and unknowable. We cannot directly compute the expected risk because we don't have access to the infinite stream of future data. Instead, we must work with the finite sample we have: the training set $\mathcal{T}$.

This motivates the definition of **empirical risk**, an estimate of the true expected risk based only on the training data. The empirical risk is simply the average loss over all training examples:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) \triangleq \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t)$$

For 0-1 loss, this quantity has a particularly natural interpretation: it is the **error rate**, the fraction of training examples that h misclassifies:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \mathbb{1}[h(\mathbf{x}) \neq t] = \frac{|\{(\mathbf{x}, t) \in \mathcal{T} \mid h(\mathbf{x}) \neq t\}|}{|\mathcal{T}|}$$

The strategy in machine learning is then straightforward: since we cannot minimize the true expected risk, we minimize the empirical risk instead. Assuming we have a hypothesis class $\mathcal{H}$ (a predefined family of candidate predictors), learning becomes an optimization problem:

$$h_{\mathcal{T}} = \mathop{\operatorname{argmin}}_{h\in\mathcal{H}} \overline{\mathcal{R}}_{\mathcal{T}}(h)$$

That is, we search for the predictor in $\mathcal{H}$ that minimizes the empirical risk on our training data. The hypothesis class $\mathcal{H}$, also called the **inductive bias**, encodes our assumptions about what kind of predictor is plausible. By restricting to $\mathcal{H}$, we make an implicit statement about the structure of the underlying problem.

However, this approach comes with a subtle but profound caveat: minimizing empirical risk does not automatically guarantee good performance on future unseen data. A predictor could fit the training set perfectly while performing poorly on new examples - a phenomenon known as **overfitting**. The gap between training risk and true risk is one of the fundamental subjects of statistical learning theory, which we will explore further as we progress.

In essence, the probabilistic framework transforms the vague goal of "learning from data" into a concrete mathematical problem: find a function in a specified class that best explains the observed data according to a chosen loss function. Yet it also reminds us that this mathematical surrogate - minimizing empirical risk - is only a proxy for what we truly care about: minimizing expected risk on unknown future data.

Inductive Bias

One of the most consequential decisions in machine learning is the choice of the hypothesis class $\mathcal{H}$. This choice embodies what we might call the learner's **inductive bias** - a set of assumptions about what kind of predictor is likely to work well for the problem at hand. When we restrict ourselves to functions in $\mathcal{H}$, we're essentially saying: "I believe that somewhere in this class lies a function that will generalize well to unseen data." The question is: how should we choose this class?

There are two competing concerns that drive this choice. First, if $\mathcal{H}$ is too small, we risk excluding the true underlying function entirely. In this case, even with infinite training data and perfect optimization, we cannot achieve low error because our class simply isn't expressive enough. Second, if $\mathcal{H}$ is too large, we gain flexibility but invite a different danger: the learning algorithm might find a function that fits the training data perfectly while performing terribly on new data. Understanding this tension requires us to think deeply about overfitting and the fundamental tradeoff between bias and variance.

Let's start with a concrete example that illustrates the overfitting problem. Consider a binary classification task where the true conditional probability of class 1 is balanced: $p_C(t = 1|\mathbf{x}) = \frac{1}{2}$ for all objects. We use the standard 0-1 loss, where we pay a penalty of 1 for any misclassification and 0 otherwise.

Now imagine we define our hypothesis class to be extremely permissive: $\mathcal{H}$ consists of all possible functions from $\mathcal{X}$ to $\{0, 1\}$. With this choice, consider the following predictor:

$$h_{\mathcal{T}}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{x} = \mathbf{x}_i \in \mathbf{X} \text{ and } t_i = 1 \\ 0 & \text{otherwise} \end{cases}$$

This function is a pure **memorizer**: it outputs 1 precisely for those training objects that were labeled 1, and outputs 0 for everything else. What happens when we evaluate it?

On the training set $\mathcal{T}$, the empirical risk is exactly zero—every training example is classified correctly by construction. The function has essentially committed the entire training set to memory. However, this apparent success is an illusion. When we apply $h_{\mathcal{T}}$ to new objects sampled from the population, the probability that any given new object appears in the training set is vanishingly small. Therefore, for most new objects, the function outputs 0, predicting the minority class. Since the two classes are equally likely, this naive strategy is no better than random guessing, achieving an expected risk of approximately $\frac{1}{2}$.

This gap between training performance and true performance is the essence of **overfitting**. The predictor has not learned any genuine pattern; it has merely memorized the training data while failing to capture any generalizable structure. In practical terms, overfitting occurs when $\mathcal{H}$ is so large that the learning algorithm can find functions that achieve perfect empirical risk despite having high expected risk.

The opposite problem arises if we choose $\mathcal{H}$ to be too small. Suppose, for instance, that the true underlying function is highly nonlinear, but we restrict ourselves to linear predictors. No matter how much training data we collect or how well we optimize within our restricted class, we cannot escape the fundamental limitation: linear functions cannot approximate nonlinear patterns. The best linear approximation might still leave substantial error on both training and test data. This scenario is called **underfitting**—the hypothesis class is too restrictive to capture the complexity of the true relationship.

With underfitting, there is no gap between training and test error because the predictor is too rigid to memorize anything. Instead, the problem is that both training and test errors remain unacceptably high. We fail to fit the training data well, and consequently, we also fail to generalize.

The Bias-Variance Tradeoff

To understand these phenomena more deeply, we need to introduce two concepts that are central to statistical learning theory: **bias** and **variance**.

Consider repeatedly generating different training sets $\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3, \ldots$ from the true distribution p_M and training a learning algorithm $\mathcal{A}$ on each one. Each training set will be slightly different (assuming the data is randomly sampled), and consequently, the algorithm will produce different predictors $h_{\mathcal{T}_1}, h_{\mathcal{T}_2}, h_{\mathcal{T}_3}, \ldots$ each time.

The **bias** of a learning algorithm measures how far the average predictor (averaged over many training sets) is from the true underlying function. Formally, bias captures systematic error—if we train on countless datasets drawn from the same distribution, do the resulting predictors tend to be systematically wrong in the same direction? High bias typically arises from restrictive hypothesis classes that cannot represent complex patterns. A linear model applied to a highly nonlinear problem will have high bias regardless of how much data you provide.

The **variance**, by contrast, measures how much the learned predictor fluctuates depending on which particular training set we happened to receive. If the hypothesis class is very large and flexible, small variations in the training set can lead to dramatically different learned functions. High variance means the algorithm is sensitive to the specific noise and quirks of the training data. This is the signature of overfitting: the algorithm has so much flexibility that it fits not only the signal but also the noise.

The fundamental insight is that these two sources of error are in tension. If we choose a small, restrictive

hypothesis class (like linear functions), we reduce variance because the class simply isn't flexible enough to chase every noise fluctuation in the data. But we pay the price of high bias because we cannot represent the true function. Conversely, if we choose a large, expressive hypothesis class (like the set of all possible functions), we can drive bias down to nearly zero because we can approximate almost any target function. But we increase variance dramatically because the algorithm can now fit the training noise.

The **bias-variance tradeoff** is the realization that we cannot simultaneously minimize both sources of error. There is an optimal hypothesis class size that balances these two concerns. Too small, and bias dominates; too large, and variance dominates. The sweet spot depends on the problem at hand: the complexity of the true underlying function, the amount of training data available, and the noise in the observations.

In practice, this tradeoff manifests in the learning curves we observe. When we train on small datasets, a very flexible model may actually perform well on both training and test data if it's able to capture the true pattern before it captures noise. But as the training set grows, flexible models begin to overfit because there's more data with different noise patterns to memorize. Meanwhile, restrictive models may never fit the training data well, no matter how much data we provide. The art of machine learning lies in choosing an inductive bias that matches the actual complexity of the underlying task.

Beyond the bias-variance tradeoff, there is a second practical concern: computational feasibility. Even if we believe that the true function lies in a certain hypothesis class $\mathcal{H}$, finding the predictor in $\mathcal{H}$ that minimizes empirical risk may be computationally intractable. Some hypothesis classes, though statistically sound, lead to optimization problems that are NP-hard or otherwise computationally prohibitive. In such cases, we might deliberately choose a smaller or different class $\mathcal{H}'$ simply to make the learning problem solvable in reasonable time. This introduces an additional layer to the inductive bias: not only do we encode our statistical beliefs about the problem, but we also pragmatically constrain ourselves based on what we can actually compute. The choice of hypothesis class thus reflects a blend of statistical theory and computational reality.